

Seminar 3

Problema Poștașului Chinez

Asist. univ. dr. Mihai Tudor

2 martie 2026

1 Breviar teoretic

Această secțiune sintetizează conceptele teoretice necesare formulării și rezolvării **Problemei Poștașului Chinez** pentru grafuri neorientate ponderate.

Definiție 1.1 (Problema Poștașului Chinez — graf neorientat ponderat). *Fie $G = (V, E)$ un graf neorientat, conex. Fiecărei muchii $e \in E$ îi este asociată o pondere*

$$w : E \rightarrow [0, \infty),$$

unde $w(e)$ reprezintă costul (de exemplu: lungime, timp sau preț) parcurgerii muchiei e o singură dată.

Un traseu închis este o succesiune de muchii care pornește și se termină în același vârf. Costul unui traseu este suma ponderilor muchiilor parcurse, fiecare muchie fiind contabilizată de atâtea ori de câte ori este parcursă.

Problema Poștașului Chinez constă în determinarea unui traseu închis de cost minim care parcurge fiecare muchie din E cel puțin o dată.

1.1 Caracterizarea structurală a soluției

Problema se reduce la transformarea grafului inițial într-un graf *eulerian* prin duplicarea unor muchii (sau, echivalent, a unor drumuri), cu cost total minim.

Teoremă 1.1 (Caracterizarea grafurilor euleriene). *Un graf neorientat conex admite un circuit eulerian dacă și numai dacă toate vârfurile sale au grad par.*

Observație 1.1. *Dacă G este eulerian, atunci orice circuit eulerian parcurge fiecare muchie exact o dată și are costul*

$$\sum_{e \in E} w(e),$$

care este evident minim, deoarece fiecare muchie trebuie parcursă cel puțin o dată.

1.2 Vârfuri de grad impar și necesitatea augmentării

Dacă graful nu este eulerian, notăm mulțimea vârfurilor de grad impar prin

$$O = \{v \in V \mid \deg(v) \text{ este impar}\}.$$

Observație 1.2 (Lema străngerii de mână). *În orice graf neorientat, numărul vârfurilor de grad impar este par. În consecință, $|O|$ este un număr par.*

Pentru a obține un circuit eulerian, trebuie să modificăm paritatea gradelor, transformând toate vârfurile din O în vârfuri de grad par. Aceasta se realizează prin duplicarea unor muchii din graf.

1.3 Efectul duplicării unui drum asupra gradelor

Propoziție 1.1. *Fie P un drum între două vârfuri distincte a și b . Dacă duplicăm toate muchiile lui P , atunci:*

- gradele vârfurilor a și b cresc cu 1;

- gradele tuturor vârfurilor interne ale lui P cresc cu 2.

Observație 1.3. Duplicarea unui drum între două vârfuri de grad impar le transformă în vârfuri de grad par, iar paritatea gradelor celorlalte vârfuri rămâne neschimbată. Prin urmare, pentru a obține un graf eulerian, vârfurile din O trebuie împerecheate două câte două, iar pentru fiecare pereche se duplică un drum care le conectează.

1.4 Formularea optimizării

Fie $O = \{v_1, \dots, v_{2k}\}$ mulțimea vârfurilor de grad impar. Pentru orice două vârfuri distincte $u, v \in O$ notăm cu $d(u, v)$ costul unui drum de cost minim între u și v în graful G . Acest cost reprezintă costul minim necesar pentru a modifica simultan paritatea gradelor vârfurilor u și v prin duplicarea muchiilor unui drum între ele.

Ideea fundamentală. Pentru a transforma toate vârfurile din O în vârfuri de grad par, trebuie să le grupăm în perechi disjuncte și să duplicăm câte un drum între vârfurile fiecărei perechi. Fiecare astfel de alegere determină un cost total al duplicărilor egal cu suma costurilor drumurilor alese.

Prin urmare, problema devine:

Cum alegem o împerechere a vârfurilor din O astfel încât suma costurilor drumurilor care le conectează să fie minimă?

Modelarea ca problemă de optimizare combinatorială

Construim un graf complet ponderat

$$K = (O, E_K),$$

definit astfel:

- mulțimea vârfurilor este O ;
- între orice două vârfuri distincte $u, v \in O$ există o muchie;
- ponderea muchiei (u, v) este $d(u, v)$.

O *împerechere perfectă* (sau *cuplaj perfect*) în K este o mulțime de muchii $M \subseteq E_K$ cu proprietățile:

- fiecare vârf din O apare în exact o muchie din M ;
- muchiile din M nu au vârfuri comune.

Costul unui cuplaj perfect M este definit prin

$$\text{cost}(M) = \sum_{(u,v) \in M} d(u, v).$$

Echivalența cu problema duplicărilor

Există o corespondență directă între:

- un cuplaj perfect M în graful complet K ;
- o alegere de drumuri în G ale căror muchii sunt duplicate pentru a obține un graf eulerian.

Mai precis:

- pentru fiecare pereche $(u, v) \in M$ duplicăm muchiile unui drum de cost minim între u și v în G ;
- costul total al duplicărilor este exact $\text{cost}(M)$;
- orice soluție validă de eulerianizare a grafului corespunde unui cuplaj perfect în K .

Observație 1.4. *Costul minim necesar pentru eulerianizarea grafului G este egal cu costul unui cuplaj perfect de cost minim în graful complet ponderat construit pe mulțimea vârfurilor de grad impar.*

1.5 Algoritmul Edmonds–Johnson

- (1) Determinăm mulțimea O a vârfurilor de grad impar.
- (2) Pentru fiecare pereche $u, v \in O$, calculăm distanța minimă $d(u, v)$ în G .
- (3) Construim graful complet ponderat $K_{|O|}$ pe vârfurile din O , în care ponderea muchiei (u, v) este $d(u, v)$.
- (4) Determinăm un **cuplaj perfect de cost minim** în $K_{|O|}$.
- (5) Pentru fiecare pereche (u, v) din cuplaj, duplicăm în G muchiile unui drum de cost minim între u și v .
- (6) În graful augmentat (care este acum eulerian), determinăm un circuit eulerian.

1.6 Costul soluției optime

Observație 1.5. *Costul minim al unui circuit poștal este*

$$\sum_{e \in E} w(e) + \sum_{(u,v) \in M} d(u, v),$$

unde M este cuplajul perfect de cost minim în graful construit pe O . Primul termen reprezintă costul obligatoriu al parcurgerii fiecărei muchii, iar al doilea termen reprezintă costul minim necesar pentru eulerianizarea grafului.

2 Algoritmul lui Dijkstra — instrument esențial

2.1 Rolul în Problema Poștaşului Chinez

În cazul ne-eulerian al Problemei Poștaşului Chinez, trebuie să determinăm costurile minime ale drumurilor dintre vârfurile de grad impar. Aceste costuri sunt utilizate ca ponderi în graful complet construit pe mulțimea O .

Pentru un vârf $s \in O$, avem nevoie de distanțele minime

$$d(s, v), \quad \forall v \in V.$$

Acestea reprezintă costul minim al unui drum între s și v în graful inițial. Algoritmul lui Dijkstra permite calculul eficient al acestor distanțe în grafuri ponderate cu ponderi nenegative.

2.2 Modelul matematic al problemei drumurilor minime

Fie $G = (V, E)$ un graf neorientat cu funcție de ponderi $w : E \rightarrow [0, \infty)$. Pentru un vârf sursă $s \in V$, dorim să determinăm pentru fiecare $v \in V$ valoarea

$$d(s, v) = \min_{P: s \rightsquigarrow v} \sum_{e \in P} w(e),$$

unde minimul se ia peste toate drumurile P care conectează s și v .

2.3 Ideea fundamentală a algoritmului

Algoritmul construiește progresiv distanțele minime pornind de la sursă. Principiul de bază este următorul:

Dacă toate muchiile au ponderi nenegative, atunci vârful cu distanța estimată minimă dintre cele încă neprocesate are deja distanța optimă.

Această proprietate permite fixarea definitivă a distanțelor într-o ordine crescătoare.

2.4 Invarianța algoritmului

Pe parcursul execuției se mențin două structuri:

- un vector $d[v]$ — cea mai bună estimare cunoscută a distanței de la s la v ;
- o mulțime S a vârfurilor *finalizate*, pentru care distanța $d[v]$ este deja optimă.

Invariantă:

Pentru orice vârf $v \in S$, valoarea $d[v]$ este egală cu distanța minimă reală de la sursă la v .

2.5 Operația de relaxare

Pentru o muchie (u, v) , dacă se cunoaște o estimare $d[u]$, putem obține o nouă estimare pentru v prin drumul care trece prin u :

$$d[u] + w(u, v).$$

Dacă această valoare este mai mică decât estimarea curentă $d[v]$, atunci actualizăm $d[v]$. Această operație se numește *relaxare* și exprimă ideea de îmbunătățire a unei soluții candidate.

2.6 Descrierea algoritmului

(1) Inițializare:

$$d[s] = 0, \quad d[v] = \infty \text{ pentru } v \neq s.$$

Mulțimea S este inițial vidă.

(2) Introducem toate vârfurile într-o coadă de priorități ordonată crescător după valorile $d[v]$.

(3) Cât timp există vârfuri neprocesate:

- (a) Extragem vârful u cu valoarea minimă $d[u]$.
- (b) Îl adăugăm în mulțimea S (distanța sa devine definitivă).
- (c) Pentru fiecare vecin v al lui u efectuăm relaxarea:

$$\text{dacă } d[u] + w(u, v) < d[v], \text{ atunci } d[v] \leftarrow d[u] + w(u, v).$$

2.7 Corectitudinea și ipoteza esențială

Corectitudinea algoritmului se bazează pe ipoteza

$$w(e) \geq 0 \quad \forall e \in E.$$

În prezența ponderilor negative, un vârf deja finalizat ar putea primi ulterior o estimare mai bună, ceea ce invalidează principiul de selecție greedy.

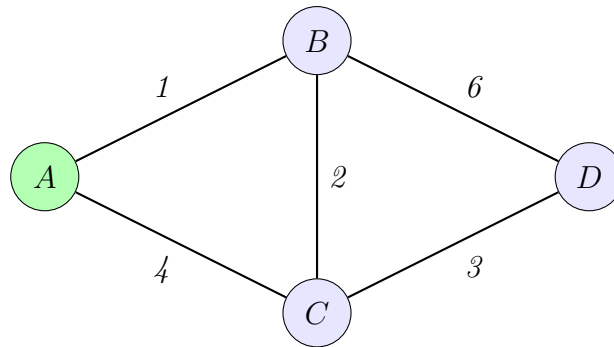
2.8 Complexitatea

Folosind o coadă de priorități implementată cu heap binar, complexitatea algoritmului este

$$O((|V| + |E|) \log |V|).$$

Pentru Problema Poștaşului Chinez, algoritmul se rulează pentru fiecare vârf din mulțimea O , obținându-se toate distanțele necesare construirii grafului complet ponderat.

Exemplu 2.1. *Urmărim pas cu pas algoritmul pe un graf simplu cu 4 vârfuri:*



Sursă: A. Inițializare: $d[A] = 0$, $d[B] = d[C] = d[D] = \infty$.

Pas	$d[A]$	$d[B]$	$d[C]$	$d[D]$	Acțiune
0	0	∞	∞	∞	Inițializare
1	0	1	4	∞	Extragem A; relaxăm B, C
2	0	1	3	7	Extragem B; relaxăm C ($1+2=3$), D ($1+6=7$)
3	0	1	3	6	Extragem C; relaxăm D ($3+3=6 < 7$)
4	0	1	3	6	Extragem D; nicio relaxare

Distanțele finale: $d[A] = 0$, $d[B] = 1$, $d[C] = 3$, $d[D] = 6$.

Drumul minim $A \rightarrow D$: $A \rightarrow B \rightarrow C \rightarrow D$ (cost $1 + 2 + 3 = 6$), nu $A \rightarrow B \rightarrow D$ (cost $1 + 6 = 7$).

2.9 Implementare Python - Algoritmul lui Dijkstra

```

1 import heapq
2
3 def dijkstra(graf, start):
4     """
5     Calculeaza distantele minime de la 'start' la toate varfurile
6     .
7     Parametri:
8         graf -- dictionar {varf: [(vecin, cost), ...]}
9         start -- varful sursa
10
11     Returneaza:
12         distante -- {varf: distanta_minima_de_la_start}
13         predecesori -- {varf: varful_anterior_pe_drumul_minim}
14                     (util pentru a reconstrui traseul)
15     """
16     # Initializam toate distantele cu infinit
17     distante = {v: float('inf') for v in graf}
18     distante[start] = 0
19
20     # predecesori[v] = de unde am ajuns la v pe drumul minim
21     predecesori = {v: None for v in graf}
22
23     # Coada de prioritati: (distanta_estimata, varf)
24     # heapq extrage intotdeauna elementul cu distanta minima

```

```

25     coada = [(0, start)]
26
27     while coada:
28         dist_curenta, u = heapq.heappop(coada)
29
30         # Daca am gasit deja un drum mai scurt, ignoram
31         if dist_curenta > distante[u]:
32             continue
33
34         # Relaxam toti vecinii lui u
35         for v, cost in graf[u]:
36             dist_noua = distante[u] + cost
37             if dist_noua < distante[v]:
38                 distante[v] = dist_noua
39                 predecesori[v] = u
40                 heapq.heappush(coada, (dist_noua, v))
41
42     return distante, predecesori
43
44
45 def reconstruieste_drum(predecesori, start, final):
46     """
47     Reconstruieste drumul minim de la start la final
48     folosind dictionarul de predecesori returnat de Dijkstra.
49     """
50     drum = []
51     varf = final
52     while varf is not None:
53         drum.append(varf)
54         varf = predecesori[varf]
55     drum.reverse()
56     # Verificam ca drumul chiar incepe din 'start'
57     return drum if (drum and drum[0] == start) else []
58
59
60 # --- Test pe exemplul de mai sus ---
61 graf_dijkstra = {
62     'A': [('B', 1), ('C', 4)],
63     'B': [('A', 1), ('C', 2), ('D', 6)],
64     'C': [('A', 4), ('B', 2), ('D', 3)],
65     'D': [('B', 6), ('C', 3)]
66 }
67
68 dist, pred = dijkstra(graf_dijkstra, 'A')
69 print("Distante de la A:")
70 for v in ['A', 'B', 'C', 'D']:
71     drum = reconstruieste_drum(pred, 'A', v)
72     print(f"  A -> {v}: cost {dist[v]}, drum: {' -> '.join(drum)}")
73
74 # Output:

```

```

75 # Distanțe de la A:
76 #   A -> A: cost 0,      drum: A
77 #   A -> B: cost 1,      drum: A -> B
78 #   A -> C: cost 3,      drum: A -> B -> C
79 #   A -> D: cost 6,      drum: A -> B -> C -> D

```

Observație 2.1. Folosim *heapq* (min-heap) din biblioteca standard Python deoarece ne permite să extragem mereu vârful cu distanța estimată minimă în timp $O(\log n)$. Complexitatea totală a algoritmului lui Dijkstra cu heap este $O((|V| + |E|) \log |V|)$.

3 Implementare CPP

3.1 Pasul 1 — Vârfuri cu grad impar

```

1 def gaseste_varfuri_impare(graf):
2     """Returneaza lista varfurilor cu grad impar."""
3     return [v for v, vecini in graf.items()
4             if len(vecini) % 2 == 1]
5
6 # Exemplu
7 graf = {
8     'A': [('B', 3), ('C', 1)],
9     'B': [('A', 3), ('C', 1), ('D', 2)],
10    'C': [('A', 1), ('B', 1), ('D', 4), ('E', 3)],
11    'D': [('B', 2), ('C', 4), ('E', 2)],
12    'E': [('C', 3), ('D', 2)]
13 }
14
15 impare = gaseste_varfuri_impare(graf)
16 print(f"Varfuri impare: {impare}")
17 # Output: Varfuri impare: ['B', 'D']
18 # (deg B = 3, deg D = 3; deg A = 2, deg C = 4, deg E = 2)

```

3.2 Pasul 2 — Distanțe minime între vârfurile impare

Rulăm Dijkstra din fiecare vârf impar. Stocăm atât distanțele cât și predecesorii (pentru a reconstrui drumurile la pasul 5).

```

1 def distante_intre_impare(graf, varfuri_impare):
2     """
3     Ruleaza Dijkstra din fiecare varf impar.
4     Returneaza doua dictionare imbricate:
5         dist[u][v] = distanta minima de la u la v
6         pred[u][v] = predecesorul lui v pe drumul minim de la u
7     """
8     dist = {}
9     pred = {}
10    for u in varfuri_impare:
11        dist[u], pred[u] = dijkstra(graf, u)

```

```

12     return dist, pred
13
14 dist, pred = distante_intre_impere(graf, impare)
15 print("Distante între varfurile impare:")
16 for u in impare:
17     for v in impare:
18         if u != v:
19             drum = reconstruieste_drum(pred[u], u, v)
20             print(f"  d({u},{v}) = {dist[u][v]}"
21                   f"    [{', -> '.join(drum)}]")
22 # Output:
23 # d(B,D) = 2    [B -> D]
24 # d(D,B) = 2    [D -> B]

```

3.3 Pasul 3–4 — Cuplajul perfect de cost minim

Cu $|O| = 2k$ vârfuri impare, numărul de cuplaje perfecte posibile este $(2k - 1)!! = (2k - 1) \cdot (2k - 3) \cdots 3 \cdot 1$. Pentru k mic (cazul uzual în probleme de seminar) le enumerăm exhaustiv.

```

1 def cuplaj_perfect_minim(varfuri_impere, dist):
2     """
3     Gaseste cuplajul perfect de cost minim prin enumerare.
4     Returneaza: (lista_de_perechi, cost_total_minim)
5
6     Numarul de cuplaje posibile pentru 2k varfuri: (2k-1)!!
7         k=1 ->  1 cuplaj
8         k=2 ->  3 cuplaje
9         k=3 -> 15 cuplaje
10    """
11    cost_min = float('inf')
12    cuplaj_optim = []
13
14    def genereaza(ramasite, cuplaj_curent):
15        nonlocal cost_min, cuplaj_optim
16        if not ramasite:
17            cost = sum(dist[a][b] for a, b in cuplaj_curent)
18            if cost < cost_min:
19                cost_min = cost
20                cuplaj_optim = list(cuplaj_curent)
21            return
22        # Fixam primul element si il cuplatm cu fiecare alt varf
23        prim = ramasite[0]
24        for i, alt in enumerate(ramasite[1:], 1):
25            cuplaj_curent.append((prim, alt))
26            genereaza(ramasite[1:i] + ramasite[i+1:],
27                      cuplaj_curent)
28            cuplaj_curent.pop()
29
30    genereaza(varfuri_impere, [])
31    return cuplaj_optim, cost_min

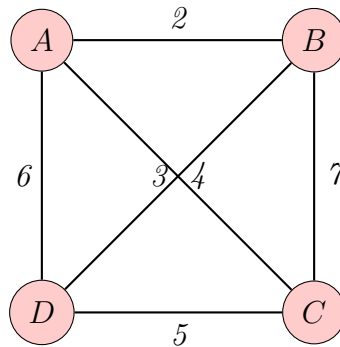
```

```

32
33 cuplaj, cost_extra = cuplaj_perfect_minim(impare, dist)
34 print(f"Cuplaj ales: {cuplaj}")
35 print(f"Cost extra: {cost_extra}")
36 # Output:
37 # Cuplaj ales: [('B', 'D')]
38 # Cost extra: 2

```

Exemplu 3.1. Caz cu 4 vârfuri impare: enumerarea celor 3 cuplaje posibile.



Toate gradele sunt 3 (impare), deci $O = \{A, B, C, D\}$. Graful este complet, distanțele minime sunt chiar costurile muchiilor directe.

Cuplaj	Perechi	Cost
1	(A, B) și (C, D)	$2 + 5 = 7$
2	(A, C) și (B, D)	$3 + 4 = 7$
3	(A, D) și (B, C)	$6 + 7 = 13$

Cuplajele 1 și 2 sunt ambele optime (cost 7). Funcția va alege primul găsit.

```

1 graf4 = {
2     'A': [('B', 2), ('C', 3), ('D', 6)],
3     'B': [('A', 2), ('C', 7), ('D', 4)],
4     'C': [('A', 3), ('B', 7), ('D', 5)],
5     'D': [('A', 6), ('B', 4), ('C', 5)]
6 }
7
8 impare4 = gaseste_varfuri_impere(graf4)
9 dist4, pred4 = distante_intre_impere(graf4, impare4)
10 cuplaj4, cost4 = cuplaj_perfect_minim(impere4, dist4)
11
12 print(f"Varfuri impere: {impere4}")
13 print(f"Cuplaj optim: {cuplaj4}")
14 print(f"Cost extra: {cost4}")
15 # Output:
16 # Varfuri impere: ['A', 'B', 'C', 'D']
17 # Cuplaj optim: [('A', 'B'), ('C', 'D')]
18 # Cost extra: 7

```

3.4 Pasul 5 — Augmentarea grafului

```

1 from copy import deepcopy
2
3 def augmenteaza(graf, cuplaj, pred):
4     """
5     Adauga in graf copii ale muchiilor de pe drumurile minime
6     corespunzatoare cuplajului. Returneaza graful augmentat.
7     """
8     g = deepcopy(graf)
9     for (u, v) in cuplaj:
10         drum = reconstruieste_drum(pred[u], u, v)
11         print(f"  Duplicam drumul {' -> '.join(drum)}")
12         for i in range(len(drum) - 1):
13             a, b = drum[i], drum[i+1]
14             # Costul muchiei a-b din graful original
15             cost = next(c for (nb, c) in graf[a] if nb == b)
16             g[a].append((b, cost))
17             g[b].append((a, cost))
18     return g
19
20 g_aug = augmenteaza(graf, cuplaj, pred)
21
22 print("\nGradele dupa augmentare:")
23 for v in g_aug:
24     print(f"  deg'({v}) = {len(g_aug[v])}")
25     print("  <- par" if len(g_aug[v]) % 2 == 0 else "  <- IMPAR!"
26         )
27 # Output:
28 # Duplicam drumul B -> D
29 #   deg'(A) = 2  <- par
30 #   deg'(B) = 4  <- par
31 #   deg'(C) = 4  <- par
32 #   deg'(D) = 4  <- par
33 #   deg'(E) = 2  <- par

```

3.5 Pasul 6 — Circuit eulerian

Reutilizăm algoritmul lui Hierholzer din Seminarul 2, adaptat la reprezentarea cu liste de perechi (vecin, cost).

```

1 def hierholzer(graf_aug, start):
2     """
3     Gaseste un circuit eulerian in graful augmentat.
4     Presupune ca graful este eulerian (toate gradele pare).
5     """
6     g = {v: list(vecini) for v, vecini in graf_aug.items()}
7     stiva = [start]
8     circuit = []
9
10    while stiva:

```

```

11     v = stiva[-1]
12     if g[v]:
13         u, cost = g[v].pop()
14         # Stergem muchia si in sens invers
15         g[u] = [(nb, c) for nb, c in g[u]
16                 if not (nb == v and c == cost)]
17         stiva.append(u)
18     else:
19         circuit.append(stiva.pop())
20
21     return circuit[::-1]
22
23 circuit = hierholzer(g_aug, 'A')
24 print("Circuit eulerian:")
25 print('  ' + ' -> '.join(circuit))
26 # Output posibil:
27 #   A -> B -> D -> B -> C -> D -> E -> C -> A

```

4 Rezolvare completă integrată

```

1 def postasul_chinez(graf, start):
2     """
3     Rezolva Problema Postasului Chinez pentru un graf neorientat
4     conex cu ponderi nenegative.
5
6     Returneaza: (circuit_optim, cost_total)
7     """
8     # --- Pasul 1: varfuri impare ---
9     impare = gaseste_varfuri_impere(graf)
10    cost_original = sum(c for v in graf
11                       for (_, c) in graf[v]) // 2
12
13    print(f"Varfuri impare ({len(impere)}): {impere}")
14    print(f"Suma tuturor muchiilor: {cost_original}")
15
16    if not impere:
17        print("Graful este deja eulerian -> nicio duplicare
18              necesara.")
19        circuit = hierholzer(deepcopy(graf), start)
20        print(f"Circuit: {' -> '.join(circuit)}")
21        print(f"Cost total: {cost_original}")
22        return circuit, cost_original
23
24    # --- Pasul 2: distante minime ---
25    dist, pred = distante_intre_impere(graf, impere)
26
27    # --- Pasurile 3-4: cuplaj perfect de cost minim ---
28    cuplaj, cost_extra = cuplaj_perfect_minim(impere, dist)
29    print(f"Cuplaj ales: {cuplaj}")
30    print(f"Cost extra: {cost_extra}")

```

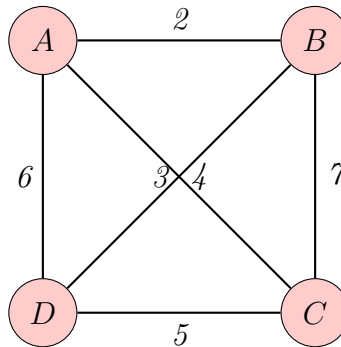
```

30
31 # --- Pasul 5: augmentare ---
32 g_aug = augmenteaza(graf, cuplaj, pred)
33
34 # --- Pasul 6: circuit eulerian ---
35 circuit = hierholzer(g_aug, start)
36
37 cost_total = cost_original + cost_extra
38 print(f"\nCircuit optim: {' -> '.join(circuit)}")
39 print(f"Cost total:      {cost_original} + {cost_extra} = {
40     cost_total}")
41 return circuit, cost_total

```

5 Exemple complete

Exemplu 5.1. *Graful K_4 — toate cele 4 vârfuri impare.*

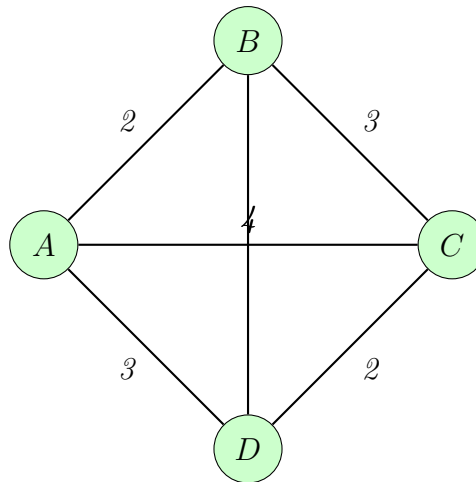


```

1 graf2 = {
2     'A': [('B', 2), ('C', 3), ('D', 6)],
3     'B': [('A', 2), ('C', 7), ('D', 4)],
4     'C': [('A', 3), ('B', 7), ('D', 5)],
5     'D': [('A', 6), ('B', 4), ('C', 5)]
6 }
7
8 postasul_chinez(graf2, 'A')
9
10 # Output:
11 # Varfuri impare (4): ['A', 'B', 'C', 'D']
12 # Suma tuturor muchiilor: 27
13 # Cuplaj ales:      [('A', 'B'), ('C', 'D')]
14 # Cost extra:      7      <- 2 + 5
15 # Duplicam drumul A -> B
16 # Duplicam drumul C -> D
17 #
18 # Circuit optim: A -> B -> A -> C -> D -> C -> ... -> A
19 # Cost total:      27 + 7 = 34

```

Exemplu 5.2. *Graf eulerian — nicio duplicare necesară.*



```

1 graf3 = {
2     'A': [('B', 2), ('D', 3), ('C', 4)],
3     'B': [('A', 2), ('C', 3), ('D', 4)],
4     'C': [('B', 3), ('D', 2), ('A', 4)],
5     'D': [('C', 2), ('A', 3), ('B', 4)]
6 }
7
8 postasul_chinez(graf3, 'A')
9
10 # Output:
11 # Varfuri impare (0): []
12 # Graful este deja eulerian -> nicio duplicare necesara.
13 # Suma tuturor muchiilor: 18
14 # Circuit: A -> B -> C -> D -> A -> C -> B -> D -> A
15 # Cost total: 18

```